

Improvement Of Data Throughput In Data-Intensive Cloud Computing Applications

Ibrahim Adel Ibrahim^{1,3}, Mostafa Bassiouni²

¹Department of Electrical & Computer Engineering

²Department of Computer Science

University of Central Florida

Orlando, USA

³University of Technology, Baghdad, Iraq

efahad@knights.ucf.edu, bassi@cs.ucf.edu

Abstract—In recent years, Hadoop framework is popularly known for providing cost-effective solutions to process large-scale data intensive applications in a distributed manner. One of the key issues that can significantly affect the performance of data-intensive cloud computing is computation load balancing among cluster nodes. Replica placement in HDFS plays a significant role in data availability and balanced utilization of clusters. In the current replica placement policy of Hadoop Distributed File System (HDFS), the replicas of data blocks cannot be evenly distributed across cluster's nodes, so the current HDFS must rely on load balancing utility for balancing replicas' distributions, which results in more time and resources consuming. In this paper, we address the load balancing problem and present an innovative replica placement policy for HDFS for data-intensive computing on massive datasets on the cloud system. It can perfectly balance the computing load among cluster's nodes in both homogeneous and heterogeneous cluster environments. Experimental results of the proposed solution confirm that, the proposed replica placement scheme gives better data nodes utilization than the default replica placement policy of Hadoop.

Keywords—Cloud Computing ; Hadoop Distributed File System; Replica Placement policy; Data Replication; Load Balance;

I. INTRODUCTION

Nowadays, storage systems are under intense pressure due to the explosive growth of data amount generated through many distributed applications, such as search engines, social networking sites, grid computing applications, data mining applications, etc. An efficient file system is required to store internet generated large data and effective handling of huge files. Cloud computing is an emerging technology that attracts data service providers offering tremendous opportunities for online distribution of services. It offers computing as a utility, sharing resources of scalable data centers [1]. End users can benefit from the convenience of accessing data and services globally, from centrally managed backups, and high computational.

MapReduce [1], designed by Google, is widely used as the most popular distributed programming model in cloud environment. The attractive feature of MapReduce is the ability to automatically parallelize a job into multiple tasks and transparently handle tasks execution by distributed processing

nodes [4]. Hadoop, an open source implementation of MapReduce, is a data management framework on large cluster of commodity machines to handle large-scale data intensive applications. Hadoop divides a large computation into small tasks and assigns them to multiple computational cluster nodes running in parallel. It manages the parallel processes on a large file, where the file is divided into many chunks distributed on clusters of inexpensive commodity computers, automatically handles failures, and hides the complexity of fault tolerance from the programmer [2]. To build such a cloud system, an increasing number of companies and academic institutions have started to rely on the Hadoop Distributed File System HDFS.

HDFS is the storage part of Hadoop framework. It is a file system designed for storing very large files reliably and stream data with high bandwidth, running on clusters on commodity hardware [2]. In a typical Hadoop cluster, terabytes of data are processed using parallel computations. The processing of jobs in a Hadoop cluster is affected by the storage mechanism of data in HDFS. Cloud computing applications that perform massive computing tasks (big data processing) offload data and tasks to data centers or powerful servers in the cloud [3]. To build such a cloud system, an increasing number of companies and academic institutions have started to rely on the Hadoop Distributed File System (HDFS) [2].

In HDFS, each data file is partitioned and stored as a sequence of data blocks, every data block is replicated to three replicas and stored on three different data nodes to improve the processing performance data, so the processes that applied on the file are performed on the data blocks. For example, in Map/Reduce jobs, data nodes perform map tasks on the file's blocks in parallel. By using the current HDFS Replica Placement Policy (RPP), the replicas are not evenly distributed over all nodes that selected to store the file. Load balancing among cluster nodes would be a great enhancement for the performance of data-intensive cloud applications. It can improve job completion time, optimize the usage of resource, maximize the throughput, minimize the retrieval time, and minimize the overload on cluster nodes.

In this paper, a fast heuristic algorithm is proposed for Replica Placement Policy (RPP) of HDFS. The new policy makes two enhancements on the current HDFS RPP. First, the

current HDFS RPP generates imbalanced distribution of replicas among cluster nodes. Therefore, HDFS provides a balancing utility to address this issue. This utility is used to analyze replicas placement and rebalancing the replicas across cluster nodes, which results in more time and resources consuming. It may reduce the performance of Hadoop applications. The proposed policy can evenly distribute the replicas to the cluster nodes while it meets all the requirements of the original HDFS RPP. As a result, there is no need to run the balancing utility that used by the current HDFS.

Second, the current HDFS RPP is developed for homogeneous cluster. It uses the same distribution pattern even if the cluster nodes differ in their processing capabilities. However, our new policy is designed to be implemented in both homogeneous and heterogeneous cluster environments, because it can achieve perfect load balancing based on the differences in processing capabilities of cluster nodes. The proposed policy is the first that addresses the load balancing problem by generating an even replica distribution to the data nodes in both homogeneous and heterogeneous clusters at the beginning of distribution and during the normal operations on data nodes.

The rest of this paper is organized as follows. Section II introduces the HDFS RPP and the related work. Section III describes the proposed data placement policy. In Section IV presents the Evaluation results, and we conclude in Section V.

II. BACKGROUND AND RELATED WORK

Hadoop has been widely adopted as a general-purpose platform for data-intensive computing. Therefore, HDFS RPP has become popular research area to improve the data partitioning and placements. In HDFS each data file is partitioned and stored as a sequence of data blocks. Every data block is replicated to three replicas and stored on three different data nodes to improve the processing performance data reliability. Assume the HDFS client runs outside the Hadoop cluster, the current HDFS RPP (as of Hadoop 2.9.2) places one replica on one node in the local rack; another on a node in a remote rack; and the third on a different node in the same remote rack. Even though HDFS RPP improves data reliability and processing performance, it has been clarified in our extensive simulations [5] that it cannot generate balanced replica distributions.

For example: a cluster of three racks, rack 0 has two data nodes, Rack 1 has three data nodes, and Rack 2 has four data nodes. Assume a file, partitioned into 34 Blocks, every block is replicated to 3 replicas, needs to be stored on the cluster nodes. The current replicas placement policy of HDFS generates unbalanced replicas distribution as shown in Figure 1. Every node gets a different number of replicas. Consequently, unbalance load distribution on cluster nodes during the parallel processes on the file blocks. Therefore, to overcome this performance issue, HDFS runs a balancing utility to analyze the load and relocating the replicas, which results in consumption of time and resources.

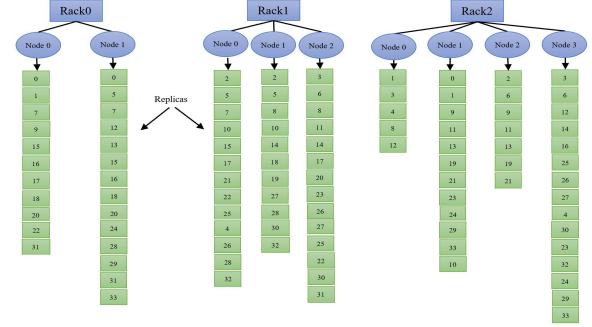


Figure 1. Replicas placement of HDFS RPP.

In our previous work [5], we eliminate the need to run the balancing utility used by RPP for HDFS. We propose an improved RPP for HDFS called Intelligent Data Placement Mechanism for Replicas Distribution in Cloud Storage Systems (IDPM), IDPM can distribute replicas evenly to cluster nodes and meet HDFS distribution requirements. However, IDPM works for homogeneous cluster environments only, while in heterogeneous cluster environments, the racks are from different Hardware generations. The balanced distribution of replicas cannot guarantee the load balancing among cluster nodes. For example, in figure 1, if the processing nodes of Rack1 are slower than the processing nodes of rack 2, the evenly distribution of replicas by using IDPM fails in balancing the load because it does not differentiate the variations in processing capabilities in cluster nodes.

Zhang et al. present Aurora in [6], a dynamic block placement mechanism, which implements several local search algorithms in HDFS. The results of Experiments show Aurora can solve the dynamic block replication problem and remarkably reduce the uneven load distribution, also it meets all the rank level reliability requirements of HDFS. Eltabakh et al. in [7], propose CoHadoop, a lightweight extension of Hadoop, it addresses the performance problem of Hadoop that it cannot colocate related data on the same set of nodes. The results of experiments on COHadoop show that only when applications need to process data from multiple files, CoHadoop can remarkably outperform Hadoop. Wai et al. in [8], address the load balance issue of the task assignment scheme in Hadoop and present an improved task assignment policy which can balance the processing load among all cluster nodes. Shabeera et al. in [9] propose a RPP for Hadoop that depends on the available bandwidth, the bandwidth between the HDFS client and cluster nodes is measured periodically, the replicas are placed on the nodes with higher bandwidth connection with the HDFS client, in order to reduce the time of data transfer. Finally, the recent research work presented in [10-12] also helps with our research in general.

III. BALANCED REPLICA PLACEMENT POLICY

The current HDFS RPP assigns two replicas to two different nodes on a randomly selected rack, and the third replica is assigned to a node on another randomly selected rack. Using this distribution, the Current HDFS RPP Generates unbalanced

replicas assignment across all cluster nodes as shown earlier in figure 1. Consequently, HDFS RPP runs a balancing utility for analyzing and rebalancing the replica placement among cluster nodes. The new proposed policy eliminates the need for this utility by balancing the distribution of replicas across cluster nodes. In homogeneous cluster nodes, the data load balancing is achieved by equalizing the number of data blocks assigned to each node, while in heterogeneous cluster, where the nodes vary in their processing capacity. The data load balance among cluster nodes cannot be achieved by merely balancing the number of data blocks assigned to each node, instead the number of blocks assigned to each node must be equivalent to the node processing capacity. The new policy generates balanced data distribution where the placement of data blocks is driven by the load on the nodes instead of selecting a randomly chosen racks and nodes as used in the current HDFS RPP.

The proposed policy evenly distributes the data blocks to cluster nodes depending on their processing speed. The node that has higher processing speed is assigned more data blocks than another node with lower processing speed in order to get better node utilization. Consequently, increase the efficiency of data processing by cluster nodes to meet the requirements of big data storage and cloud network for remote processing. For example, during Map/Reduce jobs, if map tasks are applied on the blocks of many files that stored on the same cluster nodes simultaneously, the probability of balancing the load among cluster nodes is high when the blocks have been distributed using the new policy because nodes with high processing capacity have more data blocks than slower nodes. It minimizes the chance of overloaded nodes, eventually minimizes the job completion time. Since the key to the proposed replicas placement policy is assigning replicas to nodes based on their processing capability, the new policy is called Speed-based Replica Assignment Policy (SRAP).

To deal with heterogeneous cluster environments, since the racks are usually belonging to different hardware generations, we assume the nodes on same rack are similar in their processing capability while nodes on different racks may differ in their processing capability. SRAP assigns a processing rank to the nodes. Rank of a node is the integer number of blocks it can process during a unit of time. The unit of time is the time needed by the slowest node in the cluster to process one data block. So, Rank1 is the rank of the slowest node. However, SRAP works for homogeneous cluster environments too. It is a special case of heterogeneous cluster environments where the rank of all nodes is Rank1.

A. Main Assignment Table MAT.

Let a **Cluster** consists of L racks, each of which has r_i nodes on it, where $(i = 0, 1, 2, \dots, L-1)$. Let N be the total number of all available nodes in the cluster. The nodes of each rack have a processing speed $= S_i$ where $(i = 0, 1, 2, \dots, L-1)$. Let a file with total numbers data blocks to be distributed is n , the duplication factor, D , is assumed to be three replicas for each data block in this policy. The total number of replicas to be distributed to the cluster nodes is p .

$$p = n D \quad (1)$$

The nodes of each rack have a processing rank $= K_i$, where $(i = 0, 1, 2, \dots, L-1)$. K_i is the greatest integer number less than or equal to the result of dividing the speed of nodes of rack i to the minimum speed in the cluster. It can be calculated by using the following floor function:

$$K_i = \left\lfloor \frac{S_i}{\min_{0 \leq k < L} S_k} \right\rfloor \quad (2)$$

SRAP builds a rack assignment table for every rack, it is two dimensional array with m_i columns and I rows, every node has column(s) in the rack assignment table equal its processing rank K_i . In the proposed algorithm, a node in Rack i has K_i columns allocated for it in the rack assignment table. The number of columns in assignment table of Rack i is m_i .

$$m_i = r_i K_i \quad (3)$$

The racks assignment tables are merged together to form a main table called the Main Assignment Table MAT. The number of columns in MAT is M .

$$M = \sum_{i=0}^{L-1} m_i \quad (4)$$

SRAP build the MAT for every file that is partitioned into data chunks in HDFS that need to be distributed to cluster nodes. So HDFS can use MAT of the partitioned for the file's blocks distribution among cluster nodes. The number of rows in MAT is I . It can be calculated by the ceiling function of $\frac{p}{M}$.

$$I = \left\lceil \frac{p}{M} \right\rceil \quad (5)$$

B. Blocks' numbers allocation in MAT.

SRAP builds the main assignment table MAT and distributes the blocks' numbers inside this table. Every field in MAT is corresponding an actual location in the cluster. So, HDFS can use MAT to distribute the actual blocks by placing each replica to the corresponding location referred by MAT. SRAP has p replicas to be assigned to the MAT. It starts allocating the blocks' numbers, 0 to $n-1$, into the MAT. It only allocates M replicas at a time, row after row until it reaches the last block's number.

Every column in MAT is corresponding to a node in the cluster, one node may have more than one column. Columns that dedicated for node d in rack R starts from column C and ends in column $C+K_R-1$. C can be calculated according to the following formulas:

$$C = d K_R + \sum_{i=0}^{R-1} r_i K_i \quad (6)$$

, where K_R is the rank of rack R .

Starting from Block 0, for every block's number, the algorithm picks the row and columns to allocate it in the main assignment table. The row selection starts from row 0 to row $(I-1)$. The selection of column must comply with the reliability constrains of RPP of HDFS. The column selection of SRAP

consists of two phases: rack selection phase, and node selection phase. The rack section is selected, then a free field in a column of a node inside this rack section is selected to place the replica's number on it. The rack selection, and node selection are the core of this policy. In rack selection phase, SRAP uses an index called **Rack_Index** to select two racks sections with more free fields on it. The index is continuously updated in every time a replica's number is placement in MAT. Rack_Index of a specific rack is the number of free columns in this rack section for the current row. Node selection phase starts after rack selection, in this phase, a node with more free columns from the selected rack in the current row is selected. When every node is assigned replicas equal to the number of columns it has, a new row of distribution begins.

For example, assume a cluster of three racks, Rack 0 has two nodes, Rack 1 has three nodes, and Rack 2 has four nodes. Assume the processing ranks are $(K_0) = 3$, $(K_1) = 1$, and $(K_2) = 2$. Then nodes of Rack 1 are the lowest processing capacity among the other nodes while nodes of Rack 0 are the highest processing capacity. As shown in Figure 2, SRAP generates three tables, rack 0 assignment table consists of six

columns, Rack 1 assignment table consists of three columns, and Rack 2 assignment table consists of eight columns.

The main assignment table MAT is formed from merging the three tables, it has $M = 17$ columns and I rows, where M , and I can be found according to formula (4) and (5) respectively. The main assignment table is shown in Figure 3. Let a file consists of 22 blocks, every block is replicated to three replicas. Blocks' numbers are assigned to the cells of assignment table 17 blocks at a time, with keeping the same reliability constrained of replica placement policy for HDFS. The process starts with Block 0, where two replicas of Block 0 are assigned to Rack 2, each of these two replicas are placed on different node. Rack 2 is selected at the beginning because it is the rack that has more free nodes spots. The third replica of Block 0 is assigned on another rack that has higher free nodes spots among than the other racks, it is Rack 0. When all the columns of the first row are assigned blocks' numbers, the second row starts and so on until all blocks' numbers are distributed.

Rack0 (K_0)=3					
Nodes	N0		N1		
Block No.					

Rack1 (K_1)=1			
Nodes	N0	N1	N2
Block No.			

Rack2 (K_2)=2							
Nodes	N0		N1		N2		N3
Block No.							

Figure 2. Example of Racks assignment tables.

Rack	0						1			2							
Node	0		1				0	1	2	0		1		2		3	
	Main Assignment Table																
Block No.	0	4	5	1	2	4	3	3	5	0	4	0	3	1	2	1	2
	6	7	9	5	9	11	9	10	8	6	7	6	7	8	8	10	11
	11	13	14	16	17	18	15	16	17	10	12	12	14	14	15	15	18
	16	17	19	20	22	23	21	22	22	18	19	19	20	20	21	21	23
	*****						*****			*****							

Figure 3. Main assignment table MAT for Replicas placement into three racks.

Finally, SRAP results in balanced distribution of replicas of an application in term of processing capabilities of cluster nodes for better utilization of all the cluster nodes. It makes SRAP perfect for heterogeneous cluster environment, the following is the pseudocode of the replica distribution algorithm.

Algorithm: replica distribution algorithm

Input:

A collection of L racks, each of which has r_i ($i = 0, 1, 2, \dots, L-1$) available nodes on it.

n data blocks to be distributed, which are numbered 0 through $(n - 1)$.

Rank of every cluster nodes K_i where ($i = 0, 1, 2, 3, \dots, N-1$)

Output:

$T [I][M]$: Main assignment table.

1. calculate M and I according to formula (4), and (5) respectively;
2. for ($i = 0$ to $L - 1$)
3. Rack_compete [i] = $r[i] * K[i]$; // initialize rack competition

4. for ($j = 0$ to $r[i]-1$)
5. Node_free [i][j] = $K[i]$; // indicator for the free spots in node
6. end for
7. end for
8. rack_avoid = -1
9. Block = 0; // Reset block number
10. while (Block < n) // Find the next unassigned block.
11. current_rack = Elected_rack (Rack_compete) //call function
//elected_rack to find the next rack.
12. current_node = Elected_node (current_rack) // call function
//elected_node to find the next node.
13. Count=0;
14. for ($i = 0$ to current_rack-1)
15. Count=Count + $r[i] * K[i]$ // to count the number of columns
//located before the section of current rack.
16. end for //formula no. (4)
17. Column =Count + current_node * $K[\text{current_rack}]$;
18. for ($p = 0$ to $K[\text{current_rack}]-1$)
19. if ($T [\text{iteration}[\text{current_rack}]] [\text{column}] = \text{false}$)
20. $T [\text{iteration}[\text{current_rack}]] [\text{column}+p] = \text{Block}$;

```

21.      Exit; // exit loop in case of assigned block
22.    End if;
23.  End for
24.  current_node = Elected_node (current_rack)
25.  Column = Count + current_node * K[current_rack];
      //column of the spot of selected node
26.  for (p = 0 to K[current_rack]-1)
27.    if (T [ iteration[current_rack]][column]= false)
28.      T [ iteration[current_rack]][column+p]= Block;
29.      Exit;
30.    End if;
31.  End for
32.  rack_avoid= current_rack; //avoid current rack
33.  current_rack = Elected_rack (Rack_compete)
34.  current_node = Elected_node (current_rack)
35.  for (i = 0 to current_rack-1)
36.    Count = Count + r[i] * K[i] ; // to count number of columns
      //located before the section of current rack.
37.  end for
38.  Column=Count + current_node * K[current_rack];
39.  for (p = 0 to K[current_rack]-1)
40.    if (T [ iteration[current_rack]][column]= false)
41.      T [ iteration[current_rack]][column+p]= Block;
42.      Exit;
43.    End if;
44.  End for
45.  Block=Block + 1; // take the next block
46. end while
47. END

```

During the rack selection for the third replica, the algorithm must avoid the rack that store the first and second replicas. To do so, SRAP tags the rack on which the first two replicas of current block reside as avoid_Rack to discard it during the process of finding the elected rack for the third replica. The following function to return the elected rack. This function is used two times in the main algorithm. First to place the first two replicas, and second, to place the third replica.

FUNCTION Elected_rack (Rack_compete) //returns elected Rack

```

1.  if (rack_avoid == 0)
2.    current_rack = 1; // avoid case of rack avoid is 0
3.  end if
4.  for (i = 0 to L -1)
5.    if (i == rack_avoid)
6.      Exit; // skip the current rack for the third replica
7.    end if
8.    if (iteration[i] < iteration[current_rack])
9.      current_rack = i; //use a rack with lowest iteration.
10.   end if
11.   if (Rack_compete[i] > Rack_compete[current_rack])
12.     Current_rack = i; // elect rack with more free nodes.
13.   end if
14. end for
15. Return current_rack
16. END FUNCTION

```

During the algorithm of electing a node for the selected rack, SRAP tags the node on which the first replica of current block resides as avoid_Node to discard it during the process of finding the elected node for the second replica. After each rack election SRAP assigns the replica to the elected node of this

rack. The process continues until SBRPP complete the assignment of all replicas.

FUNCTION Elected_node (Current Rack) // returns the elected node

```

1.  node_avoid = current_node
2.  for (i = 0 to r[current_rack] -1)
3.    if (i == node_avoid)
4.      Exit; // skip the current node for this replica
5.    end if
6.    if (Node_free[current_rack][i] > 0)
7.      Current_node = i;
8.      Node_free[current_rack][i] = Node_free[current_rack][i] - 1 ;
9.      Rack_compete[current_rack] =
      Rack_compete[current_rack] -1; // decrease the available // free spots
      //of nodes in the current iteration by 1.
10.   Exit; // Break loop in this case
11. end if
12. End for
13. if (Rack_compete[current_rack] == 0)
14.   Iteration[current_rack] = Iteration[current_rack] + 1 ;
15.   Rack_compete[current_rack] = r[current_rack] * K[current_rack];
      // current node multiplied by the rank of it
16.   for (j = 0 to r[current_rack]-1)
17.     Node_free[current_rack][j] = K[current_rack];
18.   End for
19. end if
20. Return current_node
END FUNCTION

```

IV. EVALUATION

The simulation results confirm that SRAP can generate perfectly even replica distributions of replicas in term of data node processing capability, due to the page limitation, we can only present the results of example simulation scenarios that are in reduced scale. Two scenarios are considered; in each scenario a simulation was conducted as shown in Table 1. The simulation of scenario one represents heterogeneous environment like the previous example in Figure 1. Three different generations of hardware exist: nodes on rack0 are of rank_3, nodes on rack1 are of rank_1, and nodes on rack2 are of rank_2. In replica distributions generated by the HDFS RPP, the replicas are not evenly distributed across the cluster nodes, and there are nodes with low computing capabilities are assigned replicas more than other nodes with high computing capabilities. It results in node overload and significant delay in data processing as shown in Figure 1. For example, node 2 of rack1 Although it has low processing speed, it has been assigned 14 data blocks, while node 0 in rack0 that has higher processing speed is assigned 9 data blocks only. On the other hand, replica distributions generated by the SRAP generates balanced replicas distribution based of the efficiency of the node as shown in Figure 4. Nodes of rack 0 have been assigned more replicas than Nodes of rack1. Scenario two represents a highly heterogeneous environment where there exist four different generations of hardware: nodes on Rack 0 are rank_1, nodes on Rack 1 are rank_2, nodes on rack 2 are rank_3, and the node on Rack 3 are rank_4. In Scenario 2, the number of replicas to be distributed to 11 nodes are 192 replicas, every column in MAT has 8 replicas. The results of scenario two are shown in Table 1.

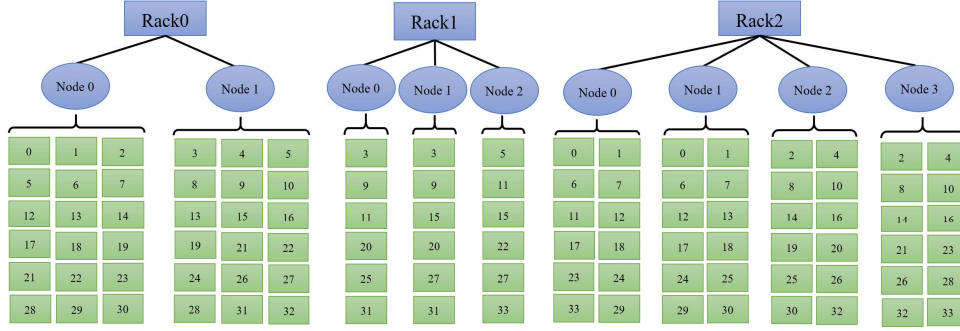


Figure 4. replica assignment generated by SRAP

TABLE 1. SIMULATION SETTINGS

Simulation Settings	Scenario One	Scenario Two
Total Number of Blocks	34	64
Duplication Factor	3	3
Total Number of Replicas	102	192
Total Number of Racks	3	4
Total Number of Nodes in each Rack	$r_i = [2, 3, 4]$	$r_i = [4, 3, 2, 2]$
rank of node in each rack	$K_i = [3, 1, 2]$	$K_i = [1, 2, 3, 4]$
Number of Replicas on One column	6	8
Minimum Number of Replicas on One Node	6	8
maximum Number of Replicas on One Node	18	32

V. CONCLUSION AND FUTURE WORK

In this paper, we have proposed a new replica placement policy for HDFS, which can remarkably improve load balancing among Hadoop cluster nodes for data-intensive applications without the need for running HDFS load balancing utility. SRAP can generate replicas distribution in a way where all cluster nodes are fully utilized, and data processing throughput is increased, as confirmed by the simulation results. The proposed policy, SRAP, is designed for heterogeneous cluster environments where hardware vary in computing capabilities are involved in cloud storage system. There are two exciting future work for the proposed policy. First, we are planning to adapt SRAP for environments where hardware with more sophisticated heterogeneity are coexist in cloud network. Second, we are planning to consider another factor, the memory, beside the processing capability of cluster node.

REFERENCES

- [1] Official Apache Hadoop Website, <http://hadoop.apache.org>, Accessed October 1, 2018.
- [2] T. White, Hadoop : The Definitive Guide, 4th ed. O'Reilly Media, March 2015.
- [3] K. Kumar and Y.-H. Lu, "Cloud computing for mobile users: Can offloading computation save energy?" Computer, 2010.
- [4] I. Ibrahim, and M. Bassiouni, "Improving MapReduce Performance with Progress and Feedback based Speculative Execution", in Proceedings of IEEE SmartCloud 2017, pp. 120-125, Columbia University, New York City, USA, November 2017.
- [5] I. Ibrahim, W. Dai, and M. Bassiouni, "Intelligent Data Placement Mechanism for Replicas Distribution in Cloud Storage Systems", in Proceedings of the 2016 IEEE International Conference on Smart Cloud (SmartCloud), pp. 134-139, New York, USA, November 2016.
- [6] Q. Zhang, S. Q. Zhang, A. Leon-Garcia, and R. Boutaba, "Aurora: Adaptive Block Replication in Distributed File Systems", in Proceedings of the 2015 IEEE 35th International Conference on Distributed Computing Systems, Columbus, USA, June 2015.
- [7] M. Y. Eltabakh, Y. Tian, F. Özcan, R. Gemulla, A. Krettek, and J. McPherson, "CoHadoop: Flexible Data Placement and Its Exploitation in Hadoop", in Proceedings of the VLDB Endowment, Vol. 4, Issue 9, pp. 575-585, June 2011.
- [8] W. Dai and M. Bassiouni, "An Improved Task Assignment Scheme for Hadoop Running in the Clouds", in Journal of Cloud Computing, Springer Publishing, Vol. 2:23, pp. 1-16, December 2013.
- [9] P. Shabeera and S. D. Madhu Kumar, "Bandwidth-aware data placement scheme for Hadoop," in Proceedings of the 2013 IEEE Recent Advances in Intelligent Computational Systems, pp. 64-67, Trivandrum, Kerala, India, 2013.
- [10] W. Dai, I. Ibrahim and M. Bassiouni, "A New Replica Placement Policy for Hadoop Distributed File System," 2016 IEEE 2nd International Conference on Big Data Security on Cloud, 2016.
- [11] W. Dai, I. Ibrahim and M. Bassiouni, "Improving load balance for data intensive computing on cloud platforms", in Proceedings of the 2016 IEEE International Conference on Smart Cloud, pp. 140-145, New York, NY, USA, November 2016.
- [12] H. Zhang, H. Huang, L. Wang, "an efficient short job optimizer on hadoop", in Proceedings of the 31st IEEE International Parallel Distributed Processing Symposium (IPDPS), 2017.